

PROGRAMMIERUNG 2

Kapitel 3: **Innere Klassen**

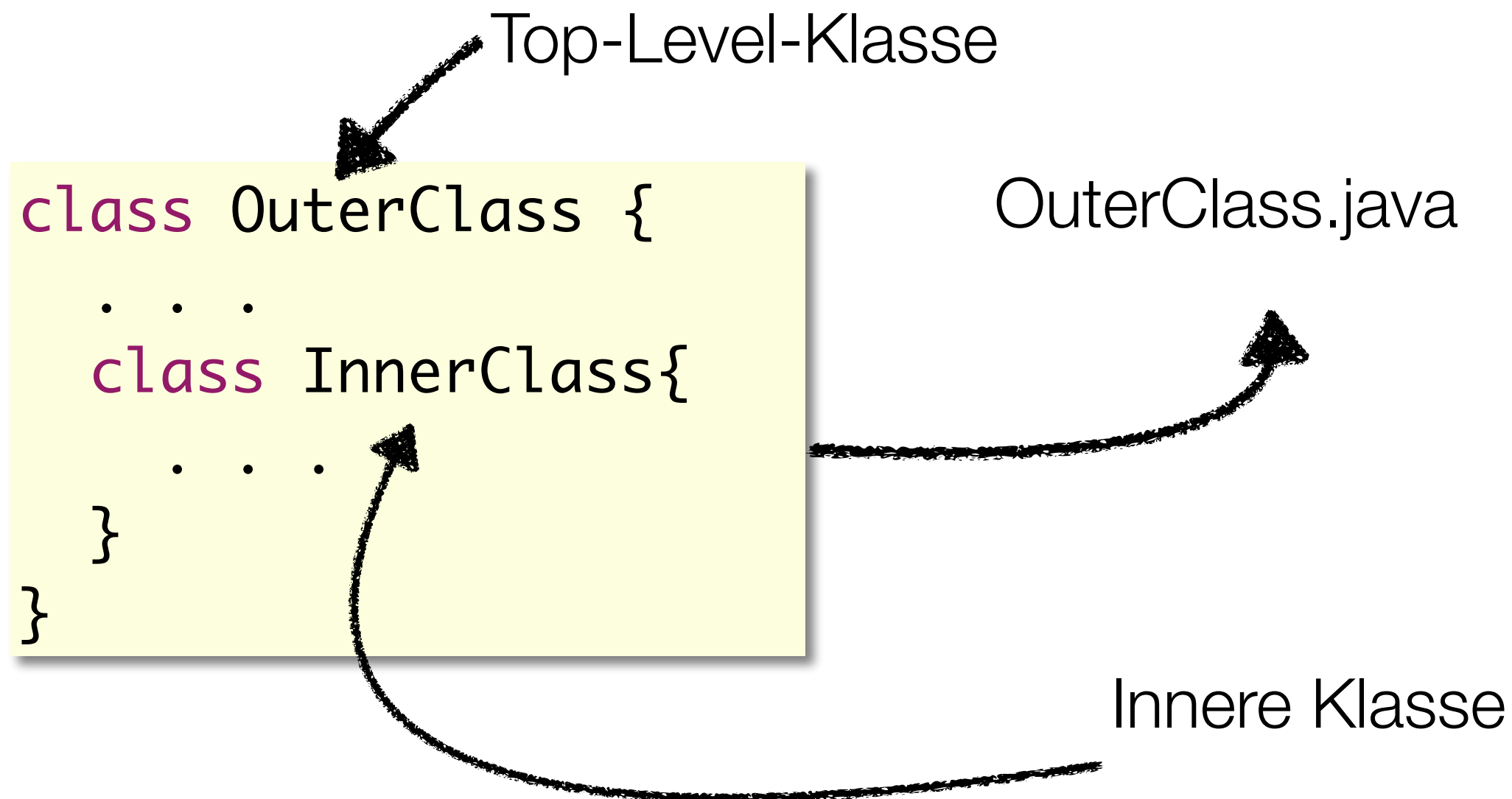


Prof. Dr. Markus Esch
htw saar

Was sind **innere Klassen**

(Nested Classes)

- Innere Klassen sind im Body einer umgebenden Klasse definiert



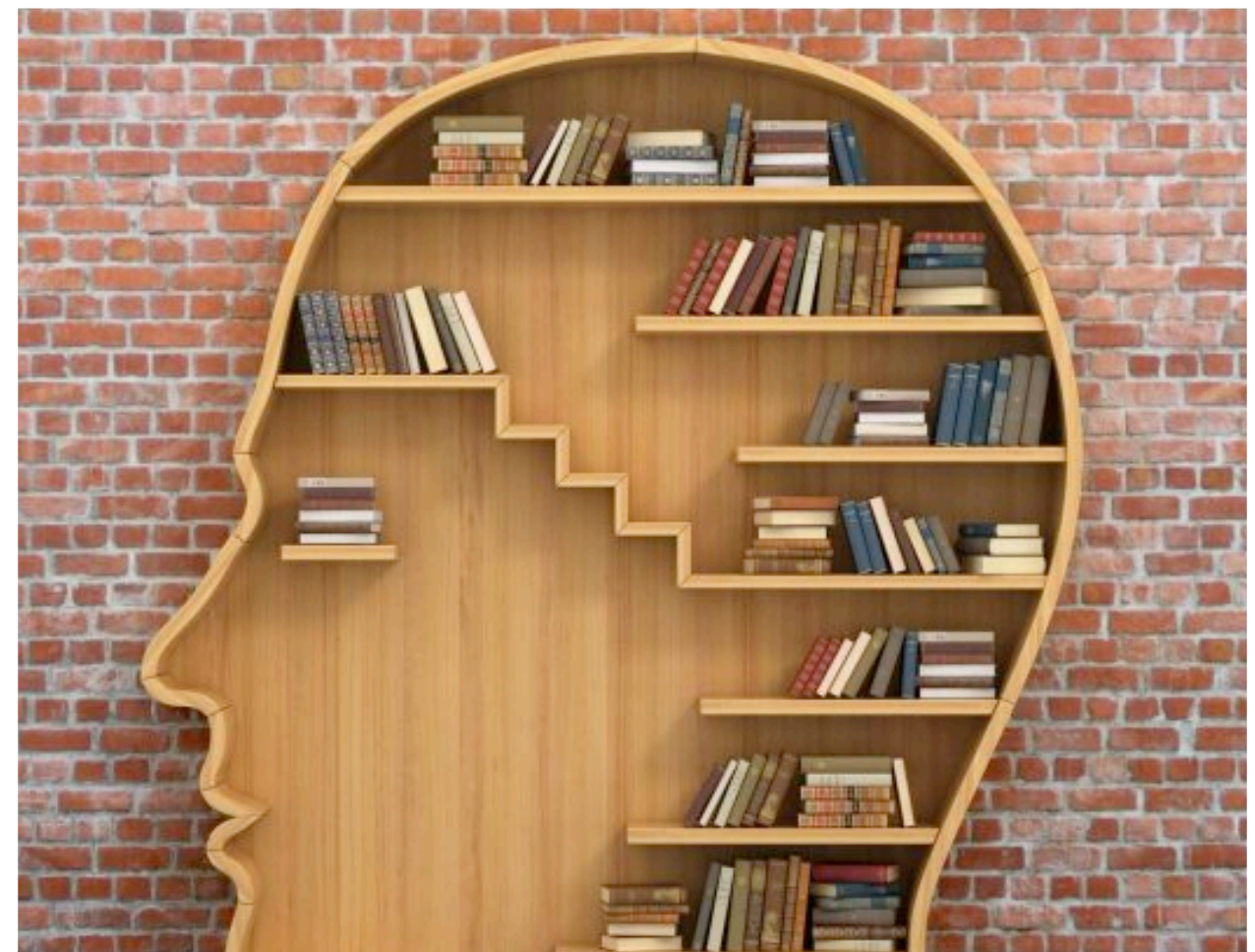
Warum innere Klassen?



Warum innere Klassen?

► Strukturierung

- Logische Gruppierung
- Definition von Hilfsklassen dort wo sie benötigt werden
- Kohäsion: Inhaltlich verwandte Aspekte an einem Ort bündeln



Warum innere Klassen?

► Kapselung

- Verbesserte Möglichkeiten Methoden und Variablen nach außen zu verbergen



Warum innere Klassen?

► Lesbarkeit

- Code steht dort, wo er verwendet wird
- sinnvoll bei “kleinen” Helferklassen



Typen innerer Klassen

- ▶ Member Inner Classes
- ▶ Static Nested Classes
- ▶ Local Classes
- ▶ Anonymous Classes



Typen innerer Klassen

- ▶ **Member Inner Classes**
- ▶ Static Nested Classes
- ▶ Local Classes
- ▶ Anonymous Classes



Member InnerClasses

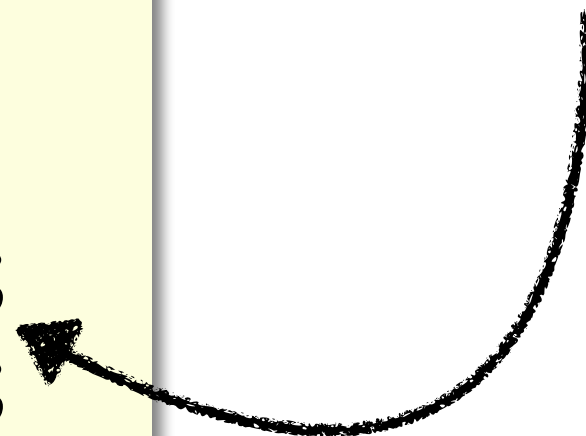
- ▶ “echte” innere Klassen
- ▶ nicht-statische innere Klasse
- ▶ voller Zugriff auf umliegende Klasse
- ▶ keine statischen Member
- ▶ Instanziierung nur über Objekt der äußeren Klasse



Member InnerClasses

```
public class A {  
    private int i;  
    public int j;  
    A(){  
        i = 42;  
        j = 420;  
    }  
    public class B{  
        public void print(){  
            System.out.println("i:"+i);  
            System.out.println("j:"+j);  
        }  
    }  
}
```

voller Zugriff auch
auf private
Member



Objekte von Member Inner Classes

```
public class A {  
    private int i;  
    public int j;  
    A(){  
        i = 42;  
        j = 420;  
    }  
    public class B{  
        public void print(){  
            System.out.println("i:"+i);  
            System.out.println("j:"+j);  
        }  
    }  
}
```

```
static void main(String[] args) {  
    A a = new A();  
    A.B b = a.new B();  
    b.print();  
}
```



Objekte der Inner Class
sind einem Objekt der
umgebenden Klasse
zugeordnet

Objekte von Member Inner Classes

```
public class A {  
    private int i;  
    public int j;  
    A(){  
        i = 42;  
        j = 420;  
    }  
    public class B{  
        public void print(){  
            System.out.println("i:" + i);  
            System.out.println("j:" + j);  
        }  
    }  
}
```

```
static void main(String[] args) {  
    A a = new A();  
    A.B b = a.new B();  
    b.print();  
    B b = new B();  
}
```

Nicht möglich!!!



Zugriff Member Inner Classes

- Eine Inner Class hat Zugriff auf **alle** umgebenden Klassen

```
public class A {  
    private int i = 21;  
    public class B {  
        private int j = 21;  
        public class C  
        {  
            private int k = i + j;  
        }  
    }  
}
```

```
static void main(String[] args) {  
    A a = new A();  
    A.B b = a.new B();  
    A.B.C c = b.new C();  
}
```


Objekte von Member Inner Classes

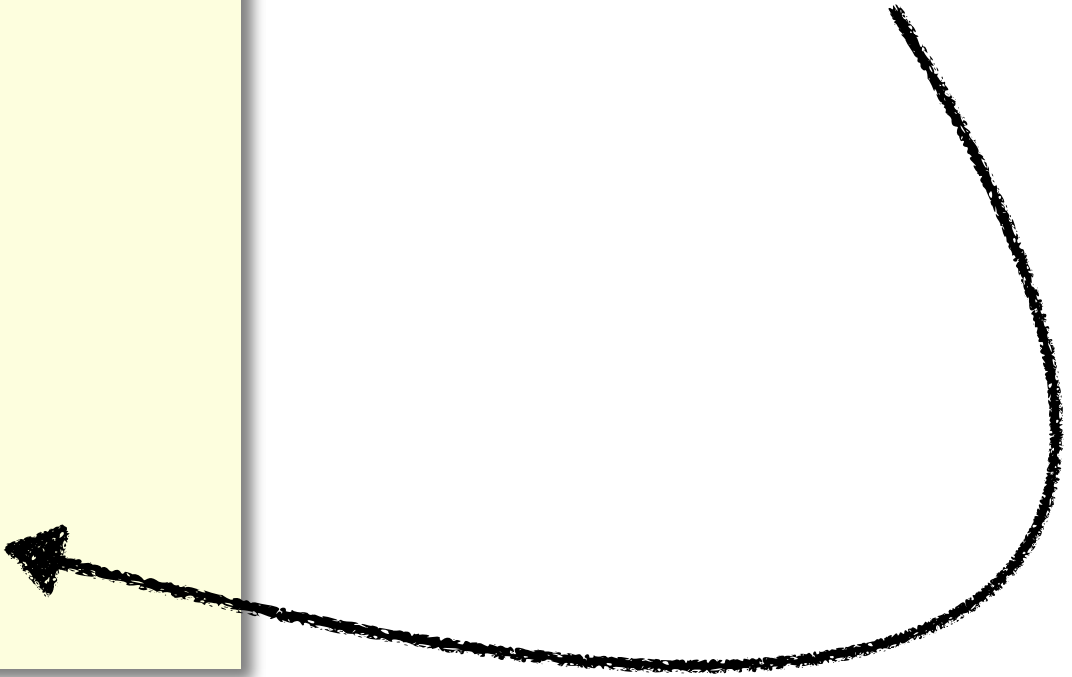
- Direkte Instanziierung aus der umgebenden Klasse

```
public class AccessInnerFromOuter {  
    int num;  
    private class Inner_Demo {  
        public void print() {  
            System.out.println("This is an inner class");  
        }  
    }  
    void display_Inner() {  
        Inner_Demo inner = new Inner_Demo();  
        inner.print();  
    }  
}
```

Objekte von Member Inner Classes

```
public class AccessInnerFromOuter {  
    private class Inner {  
        private String secret = "42";  
        private String getSecret()  
        {  
            return secret;  
        }  
    }  
    void displayInner() {  
        Inner inner = new Inner();  
        System.out.println("Inner class  
        secret: "+ inner.getSecret());  
    }  
}
```

Zugriff auf private
Member von
aussen möglich



Statische **Elemente**

- ▶ Member Inner Classes dürfen keine statischen Elemente besitzen
 - ▶ Grund: Zuordnung zu einem Objekt der umgebenden Klasse

```
public class A {  
    private static int l;  
  
    public class D{  
        static int k = l;  
    }  
}
```

Nicht möglich!!!



Member Inner Classes

kompilieren

- Compiler erzeugt eigene .class Dateien für jede Member Inner Class

```
public class X {  
    private int i = 21;  
    public class Y{  
        private int j = 21;  
        public class Z  
        {  
            private int k = i + j;  
        }  
    }  
}
```

javac X.java



X.class

X\$Y.class

X\$Y\$Z.class

Kapselung

```
public class Secret {  
    private String secret;  
}
```

```
public class SecretOwner {  
    Secret mySecret;  
    public void setSecret(Secret s)  
    {  
        mySecret = s;  
    }  
    public void beholdMySecret()  
    {  
        System.out.println("Lets behold my  
        secret: "+ . . . ? ? ?  
    }  
}
```



Kapselung

```
public class Secret {  
    private String secret;  
    public String getSecret()  
    {  
        return "Nice try!";  
    }  
}
```

```
public class SecretOwner {  
    Secret mySecret;  
    public void setSecret(Secret s)  
    {  
        mySecret = s;  
    }  
    public void beholdMySecret()  
    {  
        System.out.println("Lets behold my  
            secret: "+mySecret.getSecret());  
        System.out.println("Oh...Damn!");  
    }  
}
```



Kapselung

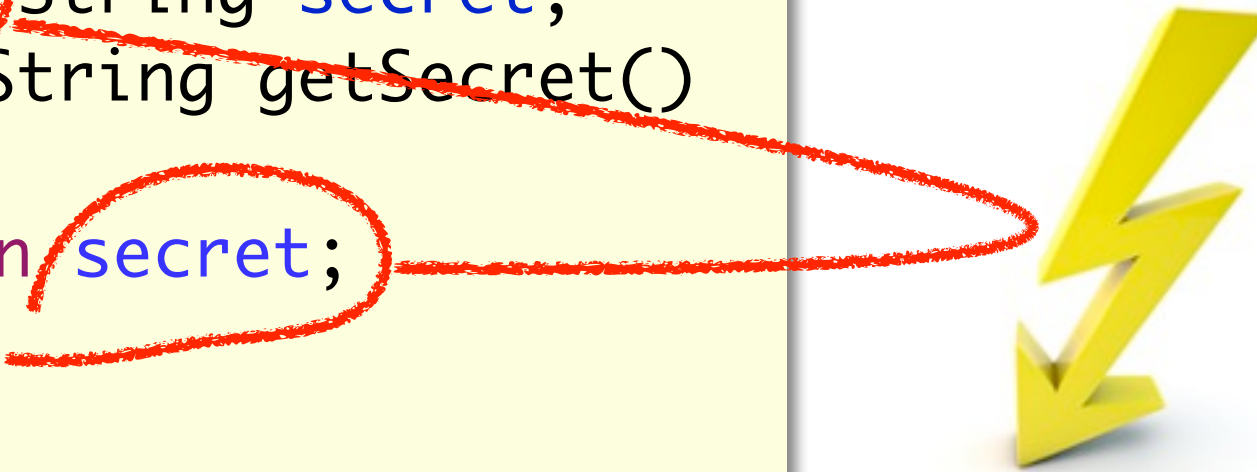
```
public class Secret {  
    private String secret;  
    public String getSecret()  
    {  
        return secret;  
    }  
}
```

```
public class SecretOwner {  
    Secret mySecret;  
    public void setSecret(Secret s)  
    {  
        mySecret = s;  
    }  
    public void beholdMySecret()  
    {  
        System.out.println("Lets behold my  
            secret: "+mySecret.getSecret());  
        System.out.println("Nice!");  
    }  
}
```



Kapselung

```
public class Secret {  
    private String secret;  
    public String getSecret()  
    {  
        return secret;  
    }  
}
```



```
public class SecretOwner {  
    Secret mySecret;  
    public SecretOwner(Secret s)  
    {  
        mySecret = s;  
    }  
    public void beholdMySecret()  
    {  
        System.out.println("Lets behold my  
            secret: "+mySecret.getSecret());  
        System.out.println("Nice!");  
    }  
}
```



Kapselung

```
public class CarefulSecretOwner {  
    Secret mySecret;  
    public CarefulSecretOwner(){  
        mySecret = new Secret();  
    }  
    public void beholdMySecret(){  
        System.out.println("Lets behold my  
        secret: "+mySecret.getSecret());  
    }  
    public class Secret{  
        private String secret = "4200";  
        private String getSecret(){  
            return secret;  
        }  
    }  
}
```



Kapselung

```
public class ProtectedSecret {  
    private String secret = "42";  
    private String getSecret(){  
        return this.secret;  
    }  
  
    public class SecretOwner{  
        public void beholdMySecret()  
        {  
            System.out.println("Lets behold  
                my secret: "+getSecret());  
        }  
    }  
}
```



Shadowing

```
public class ShadowTest {  
    public int x = 0;  
    class FirstLevel {  
        public int x = 1;  
        void methodInFirstLevel(int x) {  
            System.out.println("x = " + x);  
            System.out.println("this.x = "  
                + this.x);  
            System.out.println("ShadowTest.this.x = "  
                + ShadowTest.this.x);  
        }  
    }  
}  
  
public static void main() {  
    ShadowTest st = new ShadowTest();  
    ShadowTest.FirstLevel fl =  
        st.new FirstLevel();  
    fl.methodInFirstLevel(2);  
}
```


Shadowing

```
public class ShadowTest {  
    public int x = 0;  
    class FirstLevel {  
        public int x = 1;  
        void methodInFirstLevel(int x) {  
            System.out.println("x = " + x);  
            System.out.println("this.x = "  
                + this.x);  
            System.out.println("ShadowTest.this.x = "  
                + ShadowTest.this.x);  
        }  
    }  
}  
  
public static void main() {  
    ShadowTest st = new ShadowTest();  
    ShadowTest.FirstLevel fl =  
        st.new FirstLevel();  
    fl.methodInFirstLevel(2);  
}
```

Diagram illustrating variable resolution for the code:

- The first arrow points from the `x` in `"x = " + x` to the value `2`.
- The second arrow points from `this.x` to a question mark `?`.
- The third arrow points from `ShadowTest.this.x` to the value `0`.

Member Inner Class **Visibility**

- ▶ Innere Klassen können public, protected, package-private und private sein.

```
public class Visibility {  
    public class Public{    }  
  
    class PackagePrivate{    }  
  
    protected class Protected{    }  
  
    private class Private{    }  
}
```

Member Inner Class **Beispiel**



- **Character.java**
- **DataStructure.java**

Typen innerer Klassen

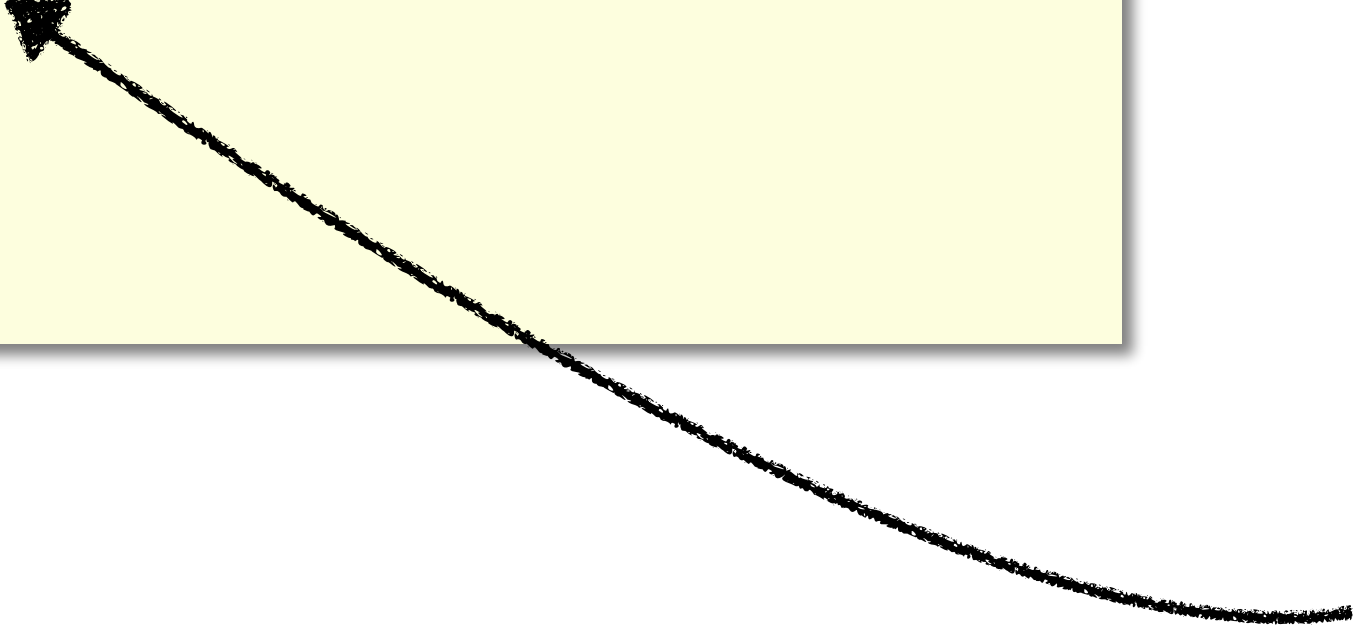
- ▶ Inner Classes
 - ▶ **Static Nested Classes**
- ▶ Local Classes
- ▶ Anonymous Classes



Static Nested Classes

- Static Nested Classes verhalten sich wie andere Top-Level-Klassen

```
class OuterClass {  
    static class StaticNestedClass{  
    }  
}
```



Wichtig:
Deklarierungen
als static

Static Nested Classes

- Verwendung analog zu Top-Level-Klassen

```
public class A {  
    public static class B {  
        public static class C {  
  
        }  
    }  
}
```

```
A a = new A();  
A.B ab = new A.B();  
A.B.C abc = new A.B.C();
```

Static Nested Classes

- Zugriff nur auf statische Variablen der umgebenden Klasse

```
public class A {  
    public int j;  
    public static int k = 42;  
    private static int l = 420;  
  
    public static class C{  
        public void print(){  
            System.out.println("l:"+l);  
            System.out.println("k:"+k);  
        }  
    }  
}
```

Static Nested Classes

- Zugriff nur auf statische Variablen der umgebenden Klasse

```
public class A {  
    public int j;  
    public static int k = 42;  
    private static int l = 420;  
  
    public static class C {  
        public void print() {  
            System.out.println("l:" + l);  
            System.out.println("k:" + k);  
            System.out.println("j:" + j);  
        }  
    }  
}
```

*Cannot make a
static reference
to a non static
field*



Static Nested Classes

- Zugriff auf private Variablen aus der umgebenden Klasse heraus

```
public class A {  
  
    public static class B{  
        private static int j;  
  
    }  
    public void a(){  
        B b = new B();  
        b.j = 42;  
        System.out.println(b.j);  
        B.j = 4200;  
        System.out.println(b.j);  
    }  
}
```

Static Nested Classes

- Zugriff auf private Variablen aus der umgebenden Klasse heraus

```
public class A {  
  
    public static class B{  
        private static int j;  
  
    }  
    public void a(){  
        B b = new B();  
        b.j = 42;  
        System.out.println(b.j);  
        B.j = 4200;  
        System.out.println(b.j);  
    }  
}
```

42

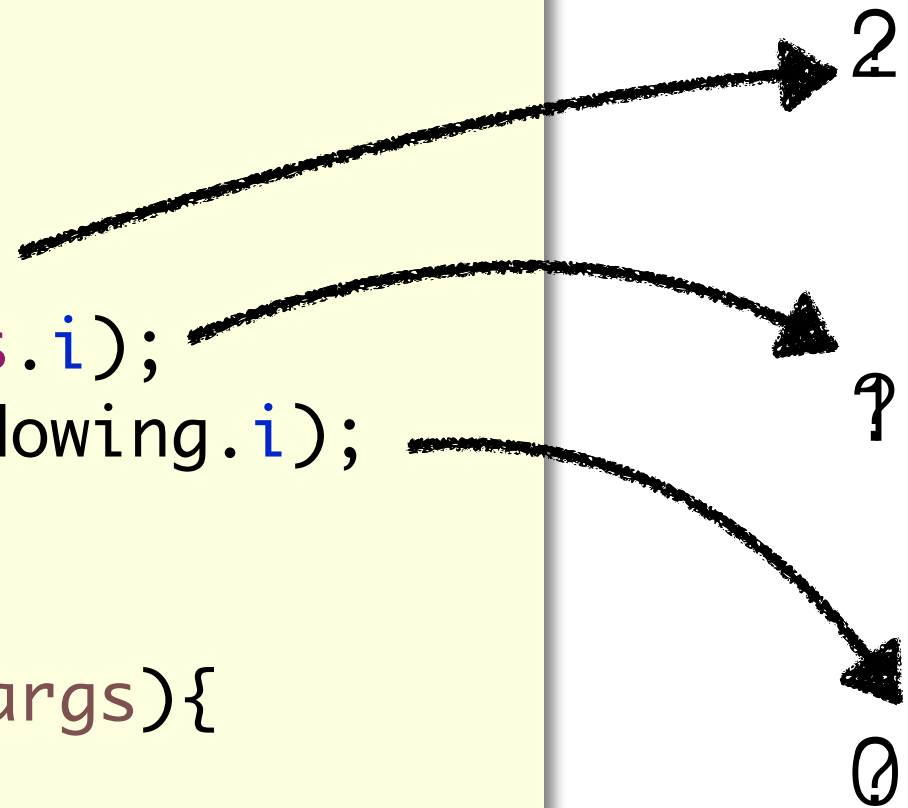
4200

Static Nested Classes: Shadowing

```
public class Shadowing {  
    public static int i = 0;  
    public static class Inner{  
        private int i = 1;  
        public void print(int i){  
            System.out.println("i: "+i);  
            System.out.println("i: "+this.i);  
            System.out.println("i: "+Shadowing.i);  
        }  
    }  
    public static void main(String[] args){  
        new Shadowing.Inner().print(2);  
    }  
}
```

Static Nested Classes: Shadowing

```
public class Shadowing {  
    public static int i = 0;  
    public static class Inner{  
        private int i = 1;  
        public void print(int i){  
            System.out.println("i: "+i);  
            System.out.println("i: "+this.i);  
            System.out.println("i: "+Shadowing.i);  
        }  
    }  
    public static void main(String[] args){  
        new Shadowing.Inner().print(2);  
    }  
}
```



Static Nested Classes **Beispiel**



- **Person.java**
- **IntBinarySearchTree.java**

Typen innerer Klassen

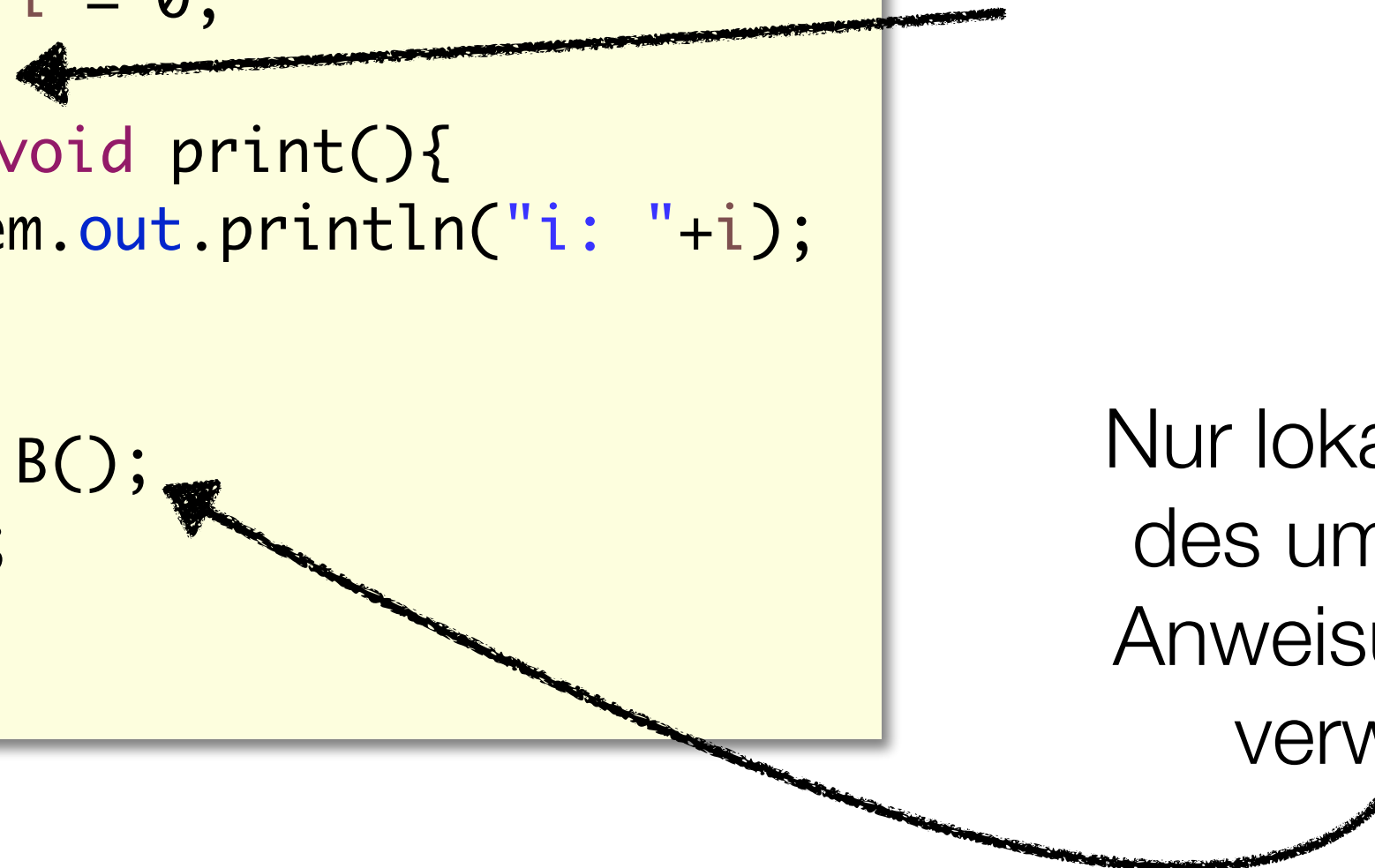
- ▶ Inner Classes
- ▶ Static Nested Classes
- ▶ **Local Classes**
- ▶ Anonymous Classes



Local Classes

Definition innerhalb
eines
Anweisungsblocks

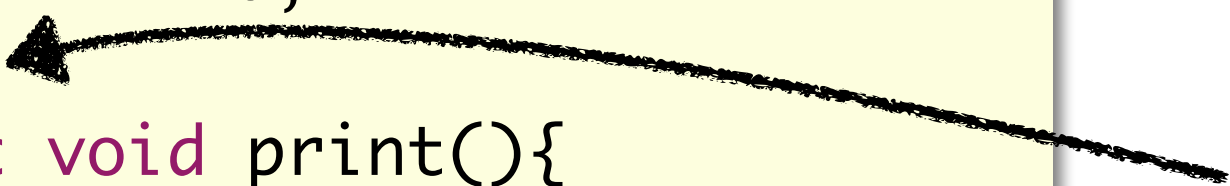
```
public class A {  
    public void doSomething(){  
        final int i = 0;  
        class B{  
            public void print(){  
                System.out.println("i: "+i);  
            }  
        }  
        B b = new B();  
        b.print();  
    }  
}
```



Nur lokal, innerhalb
des umgebenden
Anweisungsblocks
verwendbar

Local Classes: **Visibility**

```
public class A {  
    public void doSomething(){  
        final int i = 0;  
        class B{  
            public void print(){  
                System.out.println("i: "+i);  
            }  
        }  
        B b = new B();  
        b.print();  
    }  
}
```



nur Default-
Deklaration!

public, protected,
private und static
ist nicht möglich!

Local Classes: Zugriff

```
public class A {  
    public void doSomething(){  
        final int i = 0;  
        class B{  
            public void print(){  
                System.out.println("i: "+i);  
            }  
        }  
        B b = new B();  
        b.print();  
    }  
}
```

Locale Klassen
haben Zugriff auf
final Variablen des
umgebenden
Codeblocks



Local Classes: Zugriff

```
public class A {  
    public void doSomething(){  
        int i = 0;  
        class B{  
            public void print(){  
                System.out.println("i: "+i);  
            }  
        }  
        B b = new B();  
        b.print();  
    }  
}
```



seit Java 8 auch
Zugriff auf
effectively final
Variablen

Local Classes: Zugriff

```
public class A {  
    public void doSomething(){  
        int i = 0;  
        class B{  
            public void print(){  
                System.out.println("i: "+i);  
            }  
        }  
        i++;  
        B b = new B();  
        b.print();  
    }  
}
```

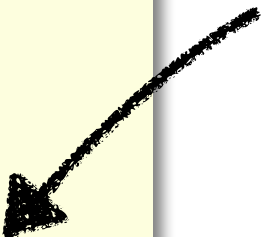
*Local variables
referenced from
an inner class
must be final or
effectively final*



Local Classes: Zugriff

```
public class A {  
    public int j = 42;  
    public void doSomething(){  
        int i = 0;  
        class B{  
            public void print(){  
                System.out.println("sum:" + i + j);  
            }  
        }  
        B b = new B();  
        b.print();  
    }  
}
```

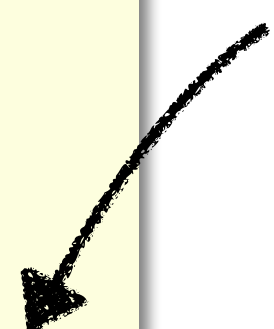
Zugriff auf Member
der umgebenden
Klasse



Local Classes: Zugriff

```
public class A {  
    public int j = 42;  
    public void doSomething(int k){  
        int i = 0;  
        class B{  
            public void print(){  
                System.out.println("sum:" + i + k);  
            }  
        }  
        B b = new B();  
        b.print();  
    }  
}
```

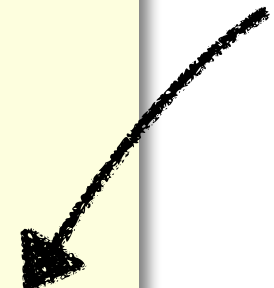
Zugriff auf
Parameter der
umgebenden
Methode



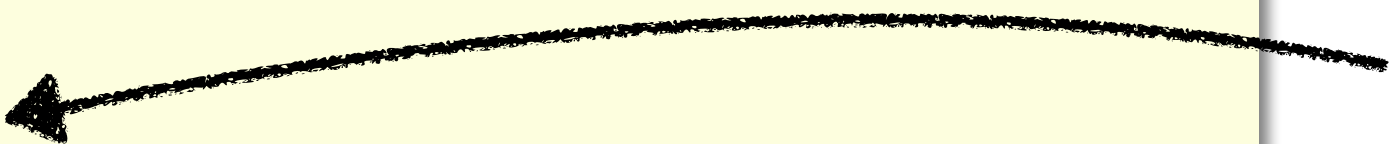
Local Classes: Zugriff

```
public class A {  
    public int j = 42;  
    public void doSomething(int k){  
        int i = 0;  
        class B{  
            public void print(){  
                System.out.println("sum:" + i + k);  
            }  
        }  
        k++;  
        B b = new B();  
        b.print();  
    }  
}
```

Zugriff auf
Parameter der
umgebenden
Methode



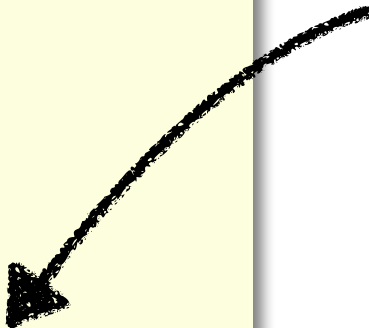
Parameter müssen
jedoch effectively
final sein!!



Statische Methoden

```
public class A {  
    public int j = 42;  
    public static int l = 42;  
  
    public static void doSomethingElse()  
    {  
        class D{  
            public void print()  
            {  
                System.out.println("l: "+l);  
            }  
        }  
    }  
}
```

Lokale Klasse in
statischen
Methoden können
nur auf **statische**
Member zugreifen



Statische Methoden

```
public class A {  
    public int j = 42;  
    public static int l = 42;  
  
    public static void doSomethingElse()  
    {  
        class D{  
            public void print()  
            {  
                System.out.println("l: "+l);  
                System.out.println("j: "+j);  
            }  
        }  
    }  
}
```

Nicht
möglich!!



Local Classes: **Schachtelung**

```
public class A {  
    public void doSomething(){  
        final int i = 0;  
        class B{  
            public void print(){  
                System.out.println("i: "+i);  
                class C{  
                    public void print(){  
                        System.out.println("2 x i: "+ 2 * i);  
                    }  
                }  
                C c = new C();  
                c.print();  
            }  
        }  
        B b = new B();  
        b.print();  
    }  
}
```

Local Classes: **Don'ts**

```
public class A {  
    public void doSomething(){  
        final int i = 0;  
        class A{  
            public void print(){  
                System.out.println("i: "+i);  
            }  
        }  
        B b = new B();  
        b.print();  
    }  
}
```

Namensgleichheit
zu einer
umgebenden
Klasse verboten!



Local Classes: **Don'ts**

```
public class A {  
    public void doSomething(){  
        final int i = 0;  
        class B{  
            public void print(){  
                System.out.println("i: "+i);  
            }  
        }  
        B b = new B();  
        b.print();  
    }  
  
    public void try(){  
        B b = new B();  
    }  
}
```

Kein Zugriff
außerhalb des
definierenden
Blocks



Local Classes: **Don'ts**

```
public class A {  
    public void doSomething(){  
        final int i = 0;  
        class B{  
            public static int j = 42;  
            public static void print(){  
                System.out.println("i: "+i);  
            }  
        }  
        B b = new B();  
        b.print();  
    }  
}
```

Keine
statischen
variablen
und
Methoden!!



Local Classes: **Don'ts**

```
public class A {  
    public void doSomething(){  
        final int i = 0;  
        class B{  
            public static final int k = 42;  
            public static int j = 42;  
            public static void print(){  
                System.out.println("i: "+i);  
            }  
        }  
        B b = new B();  
        b.print();  
    }  
}
```

Nur
statische
Konstanten
sind erlaubt



Local Classes: **Don'ts**

```
public class A {  
    public void doSomething(){  
        interface I {  
            public void print();  
        }  
        class B implements I{  
            public void print(){  
                System.out.println("i: "+i);  
            }  
        }  
        B b = new B();  
        b.print();  
    }  
}
```

*Keine lokalen
Interfaces!*



Shadowing und Local Classes

```
public class Shadowing {
    public int i = 0;
    public void doSomething(){
        int i = 1;
        class Inner{
            int i = 2;
            public void print(int i){
                System.out.println("i: "+i);
                System.out.println("i: "+this.i);
                System.out.println("i: "+Shadowing.this.i);
            }
        }
        new Inner().print(3);
    }
    public static void main(String[] args){
        new Shadowing().doSomething();
    }
}
```

Shadowing und Local Classes

```
public class Shadowing {  
    public int i = 0;  
    public void doSomething(){  
        int i = 1;  
        class Inner{  
            int i = 2;  
            public void print(int i){  
                System.out.println("i: "+i);  
                System.out.println("i: "+this.i);  
                System.out.println("i: "+Shadowing.this.i);  
            }  
        }  
        new Inner().print(3);  
    }  
    public static void main(String[] args){  
        new Shadowing().doSomething();  
    }  
}
```

3
2
0

Member **Visibility**

```
public class Shadowing {  
    public int i = 0;  
    public void doSomething(){  
        int i = 1;  
        class Inner{  
            private int i = 2;  
            public void print(int i){  
                System.out.println("i: "+i);  
                System.out.println("i: "+this.i);  
            }  
        }  
        Inner inner = new Inner();  
        inner.i = 42;  
        inner.print(inner.i);  
    }  
    public static void main(String[] args){  
        new Shadowing().doSomething();  
    }  
}
```

Zugriff auf private
Member



Local Classes: **Beispiel**



- **Phone.java**
- **EmployeeService.java**

Typen innerer Klassen

- ▶ Inner Classes
- ▶ Static Nested Classes
- ▶ Local Classes
- ▶ **Anonymous Classes**



Anonymous Classes

- ▶ Definition innerhalb eines Anweisungsblocks
- ▶ gleiche Beschränkungen wie lokale Klassen
- ▶ keine Namen
- ▶ kein Konstruktor
- ▶ Deklaration und Instanziierung gleichzeitig
- ▶ “Einweg Klassen”

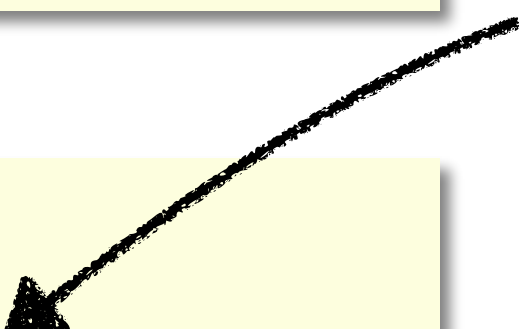


Anonymous Classes: **Syntax**

```
public interface SomeInterface {  
    public void doSomething();  
}
```

```
public class Anonymous {  
    public void someMethod(){  
        SomeInterface si = new SomeInterface(){  
            public void doSomething(){  
                . . .  
            }  
        };  
    }  
}
```

zu
implementierendes
Interface



Definition in einer
Expression



Anonymous Classes: **Syntax**

```
public class SuperClass {  
    public void doSomething(){  
    }  
}
```

```
public class Anonymous {  
    public void someMethod(){  
        SuperClass sc = new SuperClass(){  
            public void doSomething(){  
                . . .  
            }  
        };  
    }  
}
```

zu erweiternde
Klasse

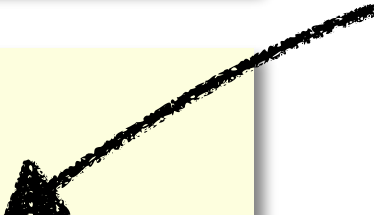


Anonymous Classes: **Syntax**

```
public class SuperClass {  
    int i;  
    SuperClass(int i){  
        this.i=i;  
    }  
    public void doSomething()  
    {  
    }  
}
```

```
public class Anonymous {  
    public void someMethod(){  
        SuperClass sc = new SuperClass(42){  
            public void doSomething(){  
                System.out.println(this.i);  
            }  
        };  
    }  
}
```

Aufruf des
Konstruktors
SuperClass(int i)



Anonymous Classes: **Syntax**

```
public class SuperClass {  
    int i;  
    SuperClass(int i){  
        this.i=i;  
    }  
    public void doSomething()  
    {  
    }  
}
```

```
public class Anonymous {  
    public void someMethod(){  
        SuperClass sc = new SuperClass(42){  
            public void doSomething(){  
                System.out.println(this.i);  
            }  
        };  
        sc.doSomething();  
    }  
}
```

Verwendung wie
ein normales
Objekt

Anonymous Classes: **Zugriff**

```
public class SuperClass {  
    public void doSomething(){  
    }  
}
```

```
public class Anonymous {  
    public void someMethod(){  
        final int j = 21;  
  
        SuperClass si = new SuperClass(){  
            public void doSomething(){  
                System.out.println(j);  
            }  
        };  
    }  
}
```

Anonyme Klassen
haben Zugriff auf
final Variablen des
umgebenden
Codeblocks



Anonymous Classes: **Zugriff**

```
public class SuperClass {  
    public void doSomething(){  
    }  
}
```

```
public class Anonymous {  
    public void someMethod(){  
        int j = 21;  
  
        SuperClass si = new SuperClass(){  
            public void doSomething(){  
                System.out.println(j);  
            }  
        };  
    }  
}
```

Zugriff auf
effectively final
Variablen



Anonymous Classes: **Zugriff**

```
public class SuperClass {  
    public void doSomething()  
    {  
    }  
}
```

```
public class Anonymous {  
    private int k = 21;  
    public void someMethod(){  
        int j = 21;  
        SuperClass si = new SuperClass(){  
            public void doSomething(){  
                System.out.println(j+k);  
            }  
        };  
    }  
}
```

Anonyme Klassen
haben Zugriff auf
Member der
umgebenden
Klasse



Anonymous Classes:

Schachtelung

```
public class Anonymous {  
    public void someMethod(){  
        SuperClass sc4 = new SuperClass(){  
            public void doSomething(){  
  
                SomeInterface si = new SomeInterface(){  
                    public void doSomething(){  
                    }  
                };  
            }  
        };  
    }  
}
```

Anonymous Classes:

Schachtelung

```
public class Anonymous {  
    public void someMethod(){  
        SuperClass sc4 = new SuperClass(){  
            public void doSomething(){  
                class LocalClass{  
                }  
            }  
        };  
    }  
}
```

Anonymous Classes: **Don'ts**

```
public class Anonymous {  
    private int k = 21;  
    public void someMethod(){  
        int j = 21;  
        SomeInterface si = new SomeInterface(){  
            public static int j = 42;  
            public static void doSomething(){  
                System.out.println(j+k);  
            }  
        };  
    }  
}
```

*Keine
statischen
variablen
und
Methoden!!*



Anonymous Classes: **Don'ts**

```
public class Anonymous {  
    private int k = 21;  
    public void someMethod(){  
        int j = 21;  
        SomeInterface si = new SomeInterface(){  
            public static final int k = 42;  
            public static int j = 42;  
            public static void doSomething(){  
                System.out.println(j+k);  
            }  
        };  
    }  
}
```

statische
Konstanten
sind erlaubt



Anonymous Classes: **Don'ts**

```
public class Anonymous {  
    public void someMethod(){  
        SuperClass sc = new SuperClass(){  
            SuperClass(int i, int j)  
            {  
                . . .  
            }  
            public void doSomething(){  
                . . .  
            }  
        };  
    }  
}
```

keine
Definition von
Konstruktoren



Anonymous Classes: **Don'ts**

```
public class SuperClass {  
    public void doSomething(){  
    }  
}
```

```
public interface SomeInterface {  
    public void doSomething();  
}
```

```
public class Anonymous {  
    public void someMethod(){  
        SomeInterface si = new SomeInterface()  
            extends SuperClass{  
                public void doSomething(){  
                    . . .  
                }  
            };  
    }  
}
```

kein extends
und kein
implements



Anonymous Classes: Shadowing

```
public class Shadowing {  
    private int i = 0;  
    public void someMethod(){  
        int i = 1;  
        SuperClass sc = new SuperClass(){  
            int i = 2;  
            public void doSomething(int i){  
                System.out.println("i: "+i);  
                System.out.println("i: "+this.i);  
                System.out.println("i: "+Shadowing.this.i);  
            }  
        };  
        sc.doSomething(3);  
    }  
    public static void main(String[] args){  
        new Shadowing().someMethod();  
    }  
}
```

3
2

0

Anonymous Classes: **Beispiel**



- **HelloWorldAnonymousClasses.java**
- **Juggler.java**
- **MessagePrinter.java**

Zusammenfassung **Vorteile**

▸ **Kapselung**

- Hilfsklassen nur dort sichtbar, wo sie benötigt werden
- verbergen von Feldern/Methoden der inneren Klassen

▸ **Namespace-Sauberkeit**

- Vermeidung überflüssiger Top-Level-Klassen
 - ➡ Übersichtlichkeit, Lesbarkeit

▸ **Direkter Zugriff**

- einfache Nutzung von Feldern/Methoden der äußeren/inneren Klasse
 - ➡ weniger Methodenparameter
 - ➡ bessere Lesbarkeit
 - ➡ Achtung! Seiteneffekte

▸ **Logische Gruppierung**

- Klassen werden kontextbezogen zusammengefasst
- Kohäsion

▸ **Reduktion von Boilerplate-Code**

- insb. bei anonyme Klassen
 - ➡ bessere Lesbarkeit



Auswahl des richtigen **Typs**?

▸ Inner Classes

- nur im Kontext einer Klasse benötigt
- enge Kopplung
- Zugriff auf Instanz-Felder benötigt

▸ Static Nested Classes

- nur im Kontext einer Klasse benötigt
- kein Zugriff auf Instanz-Felder benötigt
- Verwendung für Klassen ohne Zugriff auf äußere Instanz
 - Beispiel: Datenklassen (Data Transfer Objects), Builder, Helperklassen, Utilityklassen etc.

▸ Local Classes

- nur in lokalem Kontext benötigt
- Implementierung und Kapselung methodenspezifischer Logik
- mehr als ein Objekt

▸ Anonymous Classes

- nur in lokalem Kontext benötigt
- nur ein Objekt benötigt
- Inline-Implementierungen



Was haben Sie **gelernt**?

Erklären Sie Ihrem Sitznachbarn (45 Sekunden):

- Was ist eine Inner Class und wo verwendet man diese?
- Was ist eine Static Nested Class und wo verwendet man diese?
- Was ist eine Local Class und wo verwendet man diese?
- Was ist eine Anonymous Class und wo verwendet man diese?

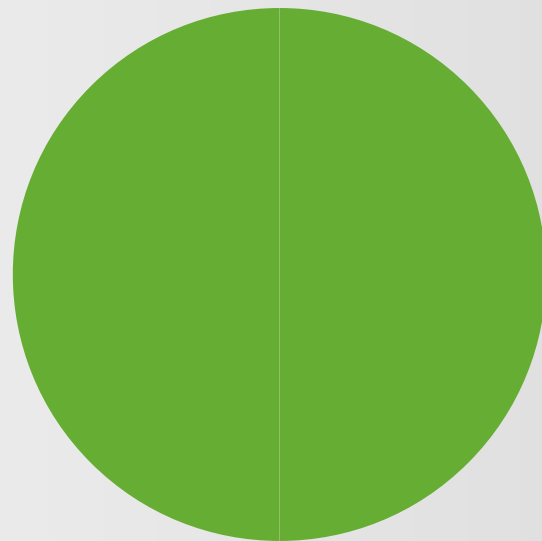


45 Sekunden

Was haben Sie **gelernt**?

Erklären Sie Ihrem Sitznachbarn (45 Sekunden):

- Was ist eine Inner Class und wo verwendet man diese?
- Was ist eine Static Nested Class und wo verwendet man diese?
- Was ist eine Local Class und wo verwendet man diese?
- Was ist eine Anonymous Class und wo verwendet man diese?



45 Sekunden

Was haben Sie **gelernt**?

Erklären Sie Ihrem Sitznachbarn (45 Sekunden):

- Was ist eine Inner Class und wo verwendet man diese?
- Was ist eine Static Nested Class und wo verwendet man diese?
- Was ist eine Local Class und wo verwendet man diese?
- Was ist eine Anonymous Class und wo verwendet man diese?

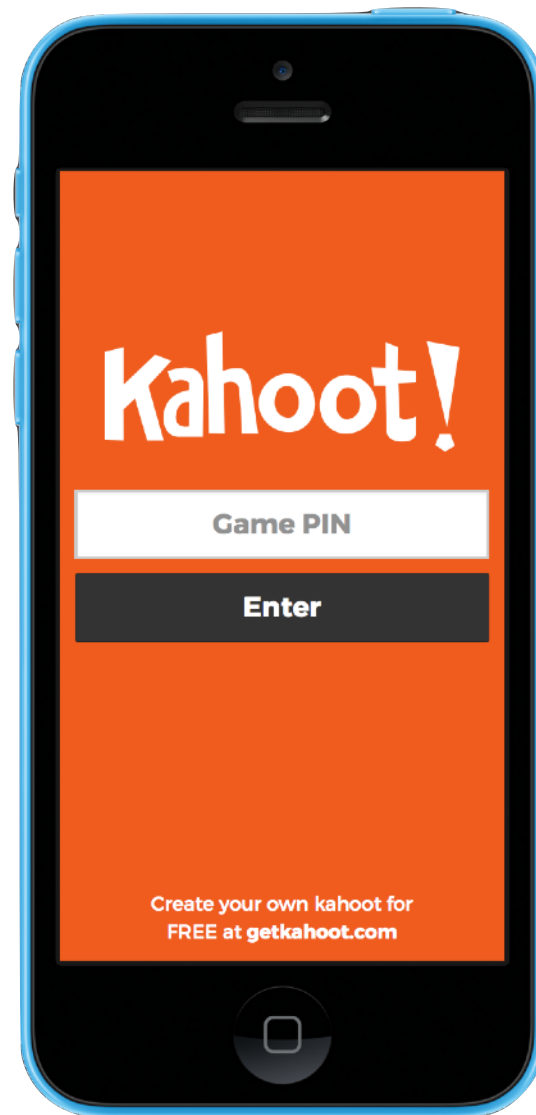


45 Sekunden

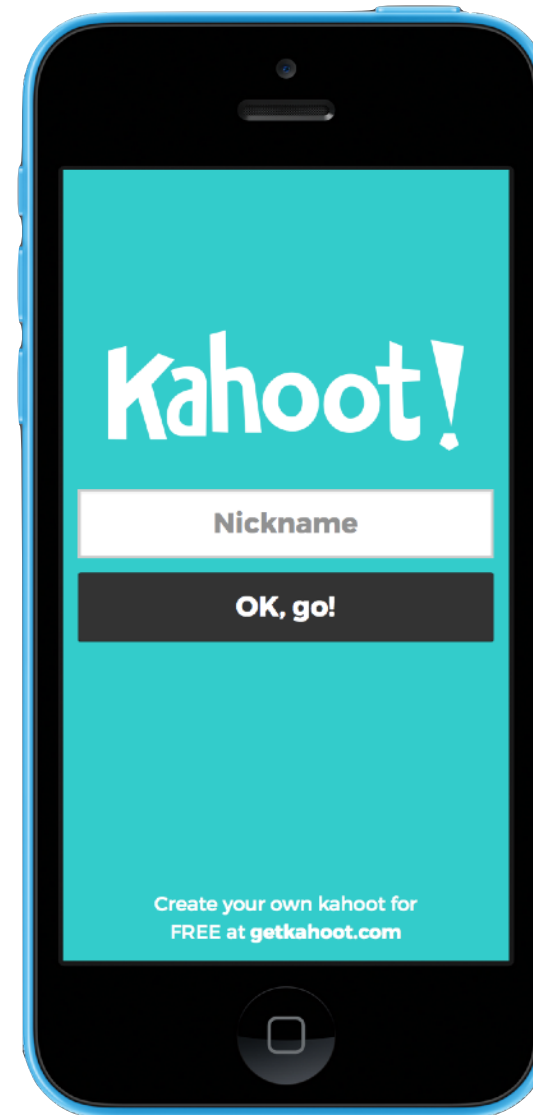
Kahoot!

go to
kahoot.it

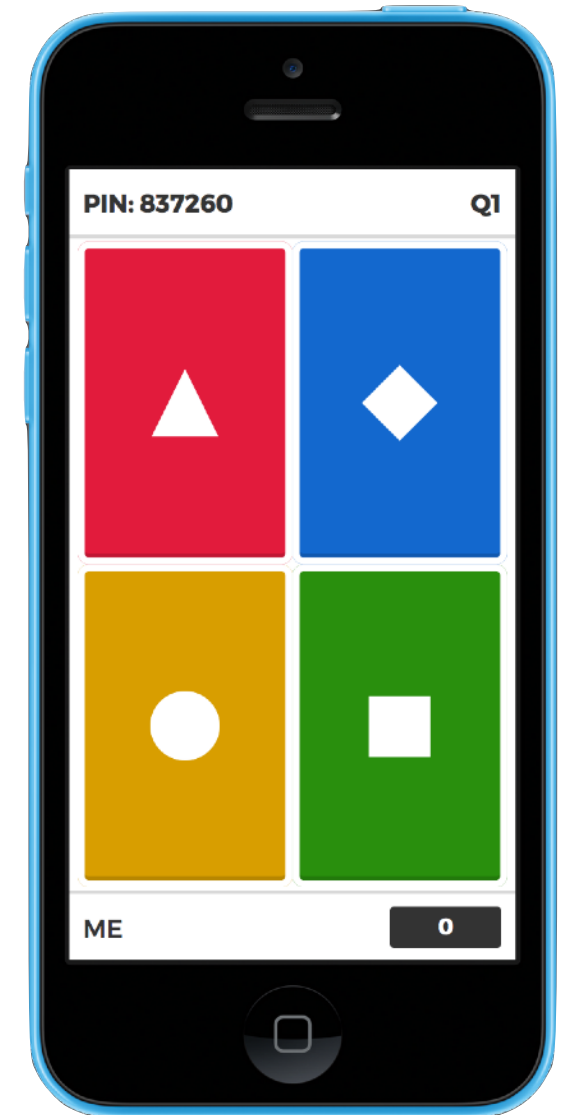
PIN
eingeben



Nickname
eingeben



mitspielen



1

2

3

4

